Autonomous Human Search System Using a Holybro S500 Drone With a Fixed Monocular Camera Based on Raspberry Pi 5

Mahija Ibad Pradipta
Department of Computer Engineering
Institut Teknologi Sepuluh Nopember
Surabaya, Indonesia
mahijapradipta86@gmail.com

Dr. Arief Kurniawan, S.T., M.T.
Department of Computer Engineering
Institut Teknologi Sepuluh Nopember
Surabaya, Indonesia
arifku@ee.its.ac.id

Dr. Eko Mulyanto Yuniarno, S.T., M.T.
Department of Computer Engineering
Institut Teknologi Sepuluh Nopember
Surabaya, Indonesia
ekomulyanto@its.ac.id

***Abstract*—The rapid advancement of Unmanned Aerial Vehicle (UAV) or drone technology presents a significant opportunity to accelerate Search and Rescue (SAR) operations in remote and inaccessible disaster areas. This research aims to design and implement an autonomous drone system based on the Holybro S500 frame for real-time victim detection utilizing edge computing on a Raspberry Pi 5 equipped with a fixed monocular webcam. The system comprehensively integrates the Pixhawk 6C Flight Controller and flight control execution via the MAVLink protocol utilizing MAVSDK Python. Based on rigorous evaluation, a fine-tuned YOLOv8n object detection model exported to ONNX format (320x320 resolution) was selected over pose estimation models due to its superior resilience in maintaining bounding box stability under varying outdoor illumination. The system implements an asynchronous multi-threaded software architecture and an isolated power regulation system using a UBEC 5A to prevent voltage sag during peak computational loads. Real-world flight tests successfully validated the persistent navigation capability of the Scout Mission through the state machine cycle: SCAN → APPROACH → DOCUMENT → DISPLACING, achieving autonomous coverage expansion without requiring interim Return-to-Launch (RTL) maneuvers upon finding a target. Furthermore, a Three-Layer Safety System effectively mitigated critical low-altitude barometric drift anomalies. Ultimately, this system successfully synergized visual perception, autonomous decision-making, and real-time operator monitoring transmission, resulting in a robust and highly efficient prototype for SAR drone operations.**

***Index Terms*—Autonomous Drone, Edge Computing, MAVSDK, Raspberry Pi 5, Search and Rescue, YOLOv8n**

I. Introduction

The continuous advancement of Unmanned Aerial Vehicle (UAV) technology provides critical capabilities for Search and Rescue (SAR) operations, especially in inaccessible disaster areas. UAVs improve mission success rates by expanding search coverage and providing real-time aerial data [1]. Rotary-wing drones, such as quadcopters, are advantageous for SAR due to their hovering capabilities and maneuverability [8].

In Indonesia, which experiences high frequencies of natural disasters [3], the adoption of drones for humanitarian operations remains limited by technological constraints in extreme conditions, lack of portable computing power, and the complexity of autonomous control [4].

Conventional drone SAR operations often rely on manual control and raw video transmission to a Ground Control Station (GCS). This introduces latency, stuttering, and signal loss in post-disaster environments with unstable networks. To address these limitations, this research proposes an edge computing approach to process visual data directly on board the aircraft.

This study details the design of a fully autonomous drone integrating a Pixhawk 6C flight controller with a Raspberry Pi 5 companion computer. This enables local execution of the YOLOv8 object detection model without relying on GCS data links. The system navigates, detects humans, approaches, and documents targets autonomously.

The primary objectives are to develop a stable hardware and power architecture, evaluate YOLOv8n ONNX inference performance on a headless OS, and construct a Persistent Multi-Target Scouting finite state machine (FSM) utilizing MAVSDK.

The scope is limited to a single Holybro S500 UAV. Edge computing relies on a single Raspberry Pi 5 (8GB) running a headless Linux OS to conserve resources. Local video recording is disabled to prevent I/O bottlenecks. Detection relies on a static monocular RGB camera identifying only the human class. Target duplicate exclusion is managed using GPS-based Haversine distance calculations.

II. Related Work

This research is supported by and built upon various comprehensive studies from previous experts relevant to the development of autonomous drones, power systems, and object detection systems for rescue missions.

A. Autonomous Search and Rescue Drone architectures

Elashaal et al. (2024) [5] developed an autonomous SAR drone using the Holybro S500, Pixhawk 2.4.8, and Raspberry Pi 4. The system utilized YOLOv4-tiny for real-time human detection, achieving a mAP of 96.44% on top-view imagery.

However, the architecture experienced high latency and low frame rates due to the computational limits of the Raspberry Pi 4. Field testing was restricted to open environments without vertical obstacles, lacking adaptive navigation logic for close-up target approaches. Furthermore, their power architecture relied heavily on generic step-down regulators, which are highly susceptible to voltage sagging when the flight controller and edge computer draw peak current simultaneously.

B. Real-Time Human Detection and Navigation for SAR

Jayalath and Munasinghe (2021) [6] integrated a TensorFlow Fast R-CNN detector with autonomous navigation on a Raspberry Pi 4 using the OkutamaAction dataset. The drone calculated GPS coordinates of victims based on geometric projection. Although achieving an 84.1% accuracy, inference latency reached 8 seconds per frame. Such severe delays are entirely insufficient for dynamic real-world SAR missions requiring control feedback frequencies above 10 Hz. The detector was also highly vulnerable to varying sunlight conditions, often losing track of subjects when transitioning between shadows and direct sunlight.

C. Target Detection, Tracking, and Avoidance System for UAVs

Varatharasan et al. (2019) [7] optimized a low-cost hardware setup using a Raspberry Pi 3B+ and an Intel Movidius Neural Compute Stick 2 (NCS2). They utilized the MAVSDK API for Offboard autonomous control, achieving system stability on low-power devices. However, the system required an external dedicated compute stick which adds mechanical payload weight and complexity to the electrical harness. Their depth estimation relied strictly on stadiametric rangefinding (estimating distance based on bounding box height), which is inherently prone to critical errors during sudden aircraft pitch or roll drifts where the geometric perspective is temporarily skewed.

III. System Design and Architecture

The methodology employed in this research relies heavily on a multi-critical Engineering Design Process framework holistically combined with Systems Engineering principles. The macro stages of this development encompass System Architecture Design, Drone Control Finite State Machine Behavior Design, Computational Electrical Planning and Design, Software Processing Pipeline Integration, and Autonomous Flight Safety Failsafe Maturation.

A. Theoretical Background of Autonomous Systems and Edge Computing

In UAV operations, edge computing shifts cognitive processing to the sensor source, eliminating bandwidth constraints and reducing decision-making latency [16]. This study utilizes YOLOv8n, a single-stage, anchor-free object detector, for its low latency on edge devices [11]. The YOLOv8 architecture replaces the traditional C3 modules with C2f (Cross Stage Partial Bottleneck with two convolutions) blocks. This topological shift significantly improves gradient flow and feature representation capabilities while maintaining a lightweight parameter count suitable for embedded Linux environments. Its structure is fundamentally divided into three primary segments: the CSPDarknet53 backbone for high-resolution feature extraction, the PANet (Path Aggregation Network) neck for multi-scale feature fusion across varying target distances, and a decoupled head that separates objectness, classification, and bounding box regression tasks. This decoupled nature is highly beneficial for accelerating the calculation of intersection over union (IoU) losses during training.

The pretrained YOLOv8n model was fine-tuned using the COCO2017 dataset, specifically filtered for the *person* class to minimize inference overhead. To optimize execution performance, the PyTorch model was exported to the ONNX (Open Neural Network Exchange) format at a 320x320 input resolution. This quantization and serialization process maximizes parallel instruction efficiency on the Raspberry Pi 5's ARM Cortex-A76 cores, drastically reducing the thermal envelope compared to standard PyTorch inference.

Communication between the companion computer and the Pixhawk 6C flight controller is handled via MAVLink, using MAVSDK Python to translate logic scripts into asynchronous dynamic vector movement commands [12] [13]. The MAVLink protocol guarantees fault-tolerant serial transmission at a baud rate of 921600. It continuously transfers critical `HEARTBEAT` signals, `LOCAL_POSITION_NED` telemetry for Cartesian positioning, and `SET_POSITION_TARGET_LOCAL_NED` messages to dictate the drone's aerodynamic behavior. The overall system architecture is detailed in Figure 1.

B. Design of the Autonomous Operation Cycle (Scout Mission)

The system's autonomous behavior is governed by a Finite State Machine (FSM) termed the *Scout Mission*. This architecture allows dynamic transitions based on environmental variables and computer vision outputs. The Scout Mission consists of four primary states (Figure 2):

- SCAN State: The drone maintains a static hover while executing a slow yaw rotation at 20 degrees per second. This maximizes the camera's observation area while the YOLO unit continuously searches for human subjects.
- APPROACH State: Upon detecting a target exceeding the confidence threshold, the drone initiates a forward approach. The velocity vector is dynamically adjusted based on the proportional difference between the target's bounding box center and the camera's actual centerpoint.
- DOCUMENT State: When the bounding box fill ratio reaches an optimal threshold and the alignment error is within safe margins, the system records GPS coordinates and saves a full-resolution image.
- DISPLACING State: Instead of executing a mandatory Return-to-Launch (RTL), the drone performs a 2-meter lateral step displacement to exit the immediate discovery area and re-enters the SCAN state for continued observation.

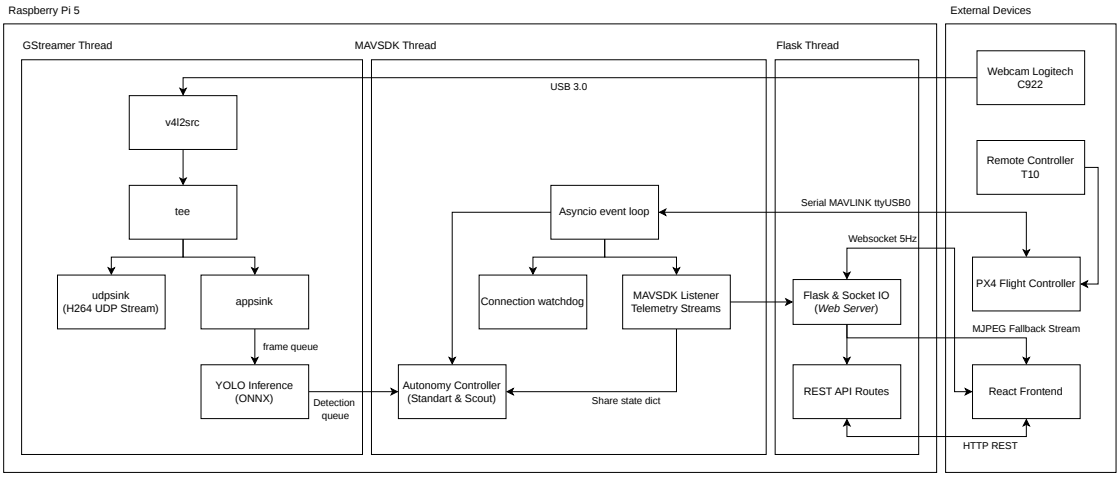


Fig. 1: Overall System Architecture Topology. This illustrates the multi-rate interaction bridge between the autonomous flight subsystem block on the Pixhawk 6C (PX4 firmware), the high-level edge computing subsystem block on the Raspberry Pi 5 companion computer, and the TCP/UDP socket broadcasting to the web GCS operator interface subsystem for state machine reporting and comprehensive mission oversight.

This architecture enables Persistent Multi-Target Scouting, allowing the drone to search continuously until critical battery levels trigger RTL. To prevent duplicate documentation of the same target, a Duplicate Exclusion Memory algorithm is implemented using the Haversine distance formula:

$$a = \sin^2\left(\frac{\Delta\phi}{2}\right) + \cos\phi_1 \cdot \cos\phi_2 \cdot \sin^2\left(\frac{\Delta\lambda}{2}\right) \quad (1)$$

$$c = 2 \cdot \text{atan2}\left(\sqrt{a}, \sqrt{1-a}\right) \quad (2)$$

$$d = R \cdot c \quad (3)$$

where R is the Earth's radius (6,371,000 m). The exclusion radius is set to 3 meters. Any new detection within this 3-meter boundary of a previously recorded target is ignored, prompting the drone to continue its search.

C. Hardware and Electrical Topology

The power system distributes energy from a single Tattu 4S Lithium Polymer (LiPo) battery (14.8V DC). Energy flows through a Holybro Power Module and Power Distribution Board (PDB) to the Electronic Speed Controllers (ESC) powering the propulsion motors.

For the Raspberry Pi 5, which demands high peak currents (up to 5A), standard power delivery methods such as GPIO-connected Mini UPS or basic step-down converters (e.g., XL4015E1) proved inadequate, leading to voltage sag, PMIC damage, or under-voltage throttling. The final robust design utilizes a 5A Universal Battery Elimination Circuit (UBEC). The UBEC connects in parallel to the main LiPo battery via an XT60 Y-splitter, providing a stable 5.1V DC supply via USB-C. This topology (Figure 3) ensures reliable power delivery, preventing voltage drops during computationally intensive inference tasks and flight maneuvers.

D. Mechanical Component Integration

The mechanical design emphasizes optimal Center of Gravity (CG) placement. Custom Fused Deposition Modeling (FDM) 3D-printed mounts, using Polylactic Acid (PLA), were fabricated for the components. This includes a tiered compartment for the LiPo battery, a ventilated housing for the Raspberry Pi 5, a UBEC protective case, and a vibration-dampened monopod for the static webcam at the lowest CG point. Figure 4 illustrates the integrated hardware subsystems.

E. Distributed Multithreading Edge Software Architecture

The software architecture utilizes multi-threading to ensure that resource-intensive AI operations do not block the critical flight control loops. The Python `threading` module is employed to partition these tasks across the Raspberry Pi 5's quad-core CPU, allowing asynchronous execution without severe global interpreter lock (GIL) bottlenecks on the heavier I/O operations:

- Main Execution Thread: Manages the Flask REST and SocketIO server for two-way WebSocket communication with the ground station. It handles the continuous transmission of JSON-formatted telemetry payloads (e.g., battery voltage, GPS coordinates, flight mode) to the operator interface at a locked 5 Hz rate.
- MAVSDK Telemetry Listener Thread: Operates asynchronously to manage flight telemetry and drive the FSM cycle. It continuously polls the Pixhawk 6C for high-frequency attitude and position updates via MAVLink. This thread evaluates the current FSM state (SCAN, APPROACH, DOCUMENT, DISPLACING) and dictates the necessary aerodynamic setpoints at a strict 20 Hz loop to ensure smooth, non-oscillating flight maneuvers.
- GStreamer Video Source Producer Thread: Handles high-speed hardware-accelerated frame capture from the USB

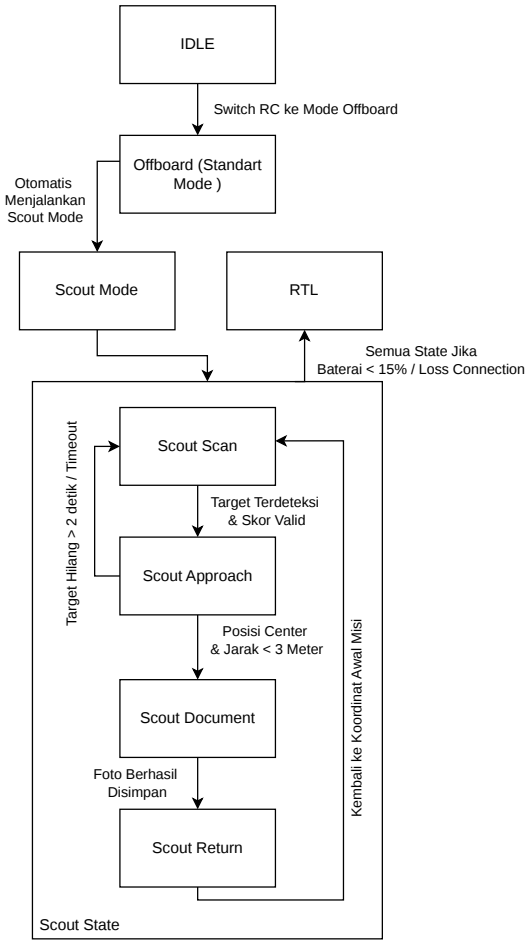


Fig. 2: Architectural action graph schematic of the *Finite State Machine* (FSM) Scout Mission, detailing the autonomy transition directions along with the handling route for failed transitions (e.g., the *Target Lost Failback* feature when the transition towards approach fails because the subject exits the camera sensor boundaries).

interface. It implements a tee-branch pipeline utilizing the `v4l2src` plugin to distribute frames simultaneously: one branch pushes raw RGB frames into a thread-safe queue for local AI inference, while the other branch encodes the feed into an H.264 stream for low-latency UDP transmission to the GCS.

- YOLO Inference Consumer Thread: Constantly dequeues the latest camera frames, applies normalization, and executes the ONNX-optimized YOLOv8n model using the ONNX Runtime engine. Upon detecting a human target, it applies Non-Maximum Suppression (NMS) and reports the highest-confidence bounding box coordinate data back to the MAVSDK thread via shared memory variables protected by threading locks.

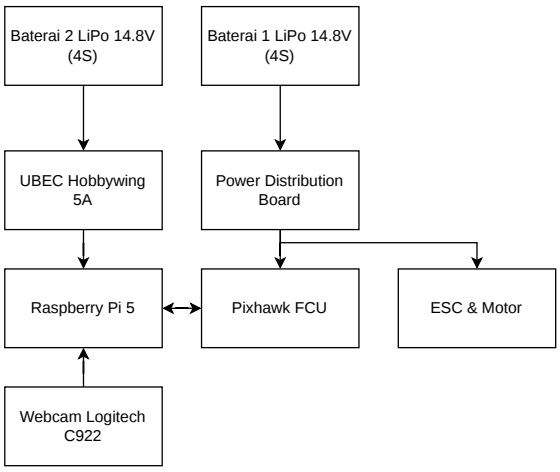


Fig. 3: Detailed Block Diagram Configuration of the Single Battery (Parallel) Electrical Load Distribution Architecture utilizing the UBEC 5A Extreme Voltage Step-Down Module.

(a) Full Assembly

Fig. 4: Final prototype implementation highlighting the payload integration, processing unit, power management, and sensor placement on the Holybro S500 frame.

F. Proportional Navigation Control Loop

To translate conceptual logic into flight maneuvers, a proportional navigation algorithm was implemented. The system periodically calculates the pixel alignment error between the target's bounding box center and the camera frame's geometric center. Because raw bounding box coordinates contain noise from wind and drone vibrations, an Exponential Moving Average (EMA) filter is applied:

$$S_t = \alpha \cdot x_t + (1 - \alpha) \cdot S_{t-1} \tag{4}$$

The smoothing factor α is set to 0.30. This filters the input error impulses x_t using the previous smoothed value S_{t-1}. The filtered alignment error is then mapped into physical setpoints.

For lateral alignment, the yaw rate ω_z (degrees per second) is defined linearly based on the horizontal pixel offset error (err_x) and a designated proportional gain (K_p):

$$\omega_z = K_p \cdot err_x \tag{5}$$

Simultaneously, the forward approach velocity v_x (meters per second) is regulated by evaluating the bounding box fill ratio A_{box} against the optimal full-screen documentation ratio A_{opt}. A damping coefficient γ limits the maximum forward pitch acceleration to prevent overshoot during approach maneuvers:

$$v_x = \gamma \cdot (A_{opt} - A_{box}) \tag{6}$$

These mathematical variables are combined to output a continuous velocity vector command for the drone via MAVSDK's `VelocityNedYaw` method. This continuous stream of coordinate instructions operates purely in the OFFBOARD flight mode, overriding manual pilot RC signals until the autonomous mission dictates a mode switch or an emergency override trigger is physically pulled by the operator.

IV. Implementation

A. Three-Layer Failsafe Safety Architecture

To mitigate the risk of sensor anomalies—specifically barometric altitude drifting common in tropical environments—a Three-Layer Failsafe Safety System was integrated into the autonomous core:

1) Altitude Floor Clamping: The minimum safe altitude is strictly constrained to 1.2 meters, preventing any autonomous command from forcing the drone to descend into the ground or jungle canopy.
2) Runtime Setpoint Rejection: At a 20 Hz loop rate, if the navigation algorithm outputs a target descent velocity (`target_alt`) that breaches the 1.2-meter limit, the command is instantly rejected and replaced with a zero-velocity vector, forcing the drone into a static hover.
3) Pre-RTL Safety Climb: When the battery reaches critical levels ($\leq 15\%$), the built-in PX4 Return-to-Launch (RTL) mode is triggered. However, the companion computer intercepts this action, stripping the current observation task and forcefully commanding a 3.0 to 5.0-second vertical climb at $v_z = -0.8$ m/s. This provides an additional 1.6m - 2.4m of elevation clearance before relinquishing control to the Pixhawk for horizontal RTL navigation, ensuring the drone clears any obstacles that might be taller than estimated due to negative barometer drift.

Additionally, an overarching heartbeat monitor guarantees that if the MAVSDK `HEARTBEAT` transmission fails to arrive at the Pixhawk within a 500 ms timeout threshold—due to a Raspberry Pi software crash or serial cable disconnection—the Pixhawk will autonomously drop out of OFFBOARD mode and trigger a fallback HOLD or RTL state. This guarantees the vehicle is never left flying blindly without algorithmic supervision.

B. GCS Operational Interface Software Layer

The monitoring interface for the Ground Control Station (GCS) is built using React 19. A two-way WebSocket communication channel (Socket.IO) handles real-time telemetry and YOLO detection bounding boxes at 5 Hz. This data overlays a low-latency H.264 GStreamer video feed transmitted via UDP, providing the operator with a comprehensive Heads-Up Display (HUD). Furthermore, a manual override is configured via the Skydroid T10 remote controller. Toggling the designated RC switch immediately terminates the OFFBOARD control loop, reverting the drone to manual Position Control (POSCTL) mode.

V. Experimental Results

The system was evaluated through empirical testing of both static components and in-flight operations over five flight sessions. Tests assessed power stability, algorithm accuracy, end-to-end latency, and autonomous navigation reliability.

A. System Power Integrity and OS Optimization

Testing the power architecture on the Raspberry Pi 5 revealed that conventional Mini UPS modules or standard buck converters (e.g., XL4015E1) failed under the high computational load, causing PMIC damage or *under-voltage* warnings leading to system reboots. The final integration of a 5A UBEC directly connected to the main LiPo battery successfully provided a stable 5.1V DC supply, entirely eliminating *brownout* incidents even during peak inference loads.

Additionally, executing the system on a headless Linux OS (without a GUI) reduced idle RAM consumption from $\sim$1,200 MB to $\sim$420 MB. This optimization prevented frame dropping and latency spikes, enabling a stable processing pipeline.

B. Detection Algorithm Architecture Evaluation

An empirical comparison was conducted between the baseline YOLOv8n model and its pose-estimation variant (YOLOv8n-pose). While the pose-estimation model successfully identified skeletal keypoints, it exhibited severe bounding box flickering under varying outdoor illumination and was highly susceptible to motion blur caused by the drone's maneuvers. Consequently, the baseline YOLOv8n bounding box detector was selected for its robustness and consistent accuracy during flight.

Further optimization focused on determining the optimal ONNX tensor input resolution. The computational benchmarking on the Raspberry Pi 5 measured the impact of three different tensor dimensions on the ONNX Runtime engine. As detailed in Table I, a 640x640 resolution resulted in an unacceptable latency of 171.22 ms (5.84 FPS), posing a severe risk to the real-time closed-loop control architecture. Conversely, a 256x256 resolution yielded 35.54 FPS but suffered from significant precision loss at distances exceeding 10 meters. Ultimately, the 320x320 resolution provided an optimal balance, reducing inference latency to 42.17 ms ($\sim$23.72 FPS) while maintaining sufficient accuracy for target detection.

C. YOLOv8n Transfer Learning and Fine-Tuning

The selected YOLOv8n model was fine-tuned over 50 epochs using a filtered COCO2017 dataset containing 123,287 images limited strictly to the *person* class. The training achieved convergence at the 40th epoch (Figure 5). Validation

TABLE I: Benchmarking of YOLOv8n ONNX Input Resolutions on Raspberry Pi 5

Model Variant	Input Size	Output Tensor	Avg FPS	Latency (ms)
256n	256x256 px	[1, 300, 6]	$\sim$ 35.54	$\sim$ 28.13
320n	**320x320 px**	**[1, 300, 6]**	$\sim$ **23.72**	$\sim$ **42.17**
640n	640x640 px	[1, 300, 6]	$\sim$ 5.84	$\sim$ 171.22

metrics (Table II) confirm high performance: a Precision of 0.809, Recall of 0.646, and mAP@50 of 0.757. The peak F1-Score of 0.72 at a confidence threshold of 0.224 ensured reliable detection capability in the field.

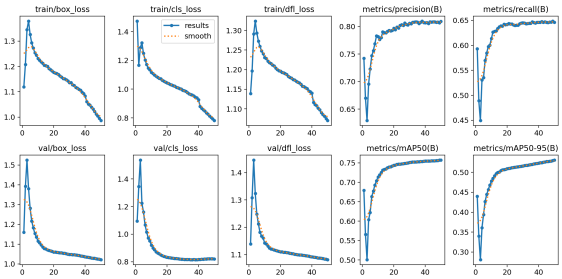


Fig. 5: Training metrics progression for the YOLOv8n model over 50 epochs.

TABLE II: Evaluation Metrics for the Fine-Tuned YOLOv8n Model

Metric	Value
Precision	0.809
Recall	0.646
mAP@50	0.757
mAP@50-95	0.531
Peak F1-Score	0.72 at Threshold 0.224

D. Autonomous Flight Testing: FSM Scout Mission

The Persistent Multi-Target Scouting FSM was validated across 38 approaching maneuver triggers during flight. The transition from SCAN to APPROACH operated flawlessly, recording a 100% success rate. Furthermore, the drone successfully completed the DOCUMENT phase 30 times. Only 8 instances resulted in tracking failures, primarily due to targets moving rapidly outside the camera's narrow Field of View (FOV) during the APPROACH phase. In these cases, the autonomy safely executed a failback loop, immediately returning to the SCAN state without system failure. The FSM execution sequence is visualized in Figure 6.

E. In-Flight Visual Catch Distance Accuracy Test

The endurance and tracking performance of the object detection algorithm were evaluated during stable hovering conditions at altitudes between 1 and 2 meters (AGL). A test subject moved linearly away from the drone, increasing distance from 5 to 20 meters. Detection accuracy and bounding box stability were measured against the distance threshold. At ranges $\leq$ 15 meters, the system maintained a 100% detection success rate with consistent bounding box area ratios. However, as the distance approached 20 meters, the target resolution on the 320x320 input grid became insufficient, resulting in increased tracking uncertainty and detection failures. The detection rate dropped to 45.5% at the 20-meter mark, identifying this distance as the functional limit of the current optical configuration.

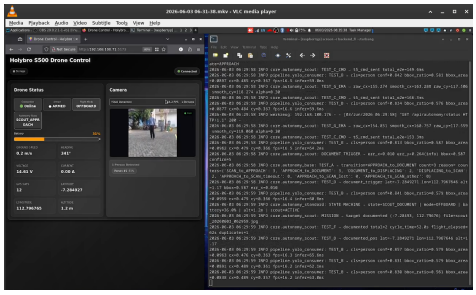
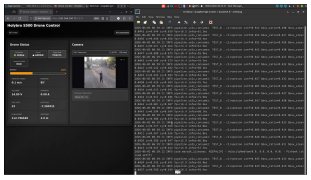
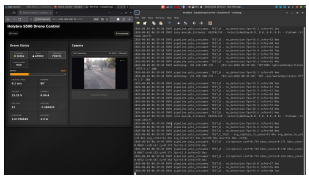


Fig. 6: In-flight operational log of the Finite State Machine transitions during the Scout Mission.



(a) Stable detection at 5 meters with 100% reliability.



(b) Degraded detection at 20 meters (45.5% reliability).

Fig. 7: Comparison of in-flight visual tracking capability at varying distances, illustrating the resolution limitations of the 320x320 input grid at extended ranges.

F. End-to-End Latency and Navigation Accuracy

Latency tests across 2,096 samples over five flight sessions measured the total time from frame extraction to the delivery of a flight speed setpoint to the Pixhawk. The overall average latency was 188.04 ms. The median (P50) latency was 164.65 ms, and the 95th percentile (P95) was 208.27 ms. This ensures the multi-threaded architecture can supply control feedback faster than a 5 Hz rate, meeting the stringent requirements for real-time mission execution without destabilizing the drone's aerodynamic stance. The detailed breakdown of the computational stages contributing to the total end-to-end latency is presented in Table III.

Navigation accuracy was assessed via the horizontal alignment error during 30 documentation phases. The system achieved a bounding box fill ratio of 0.389 and an average alignment error of 0.049. A 2.0-meter displacement shift demonstrated minimal deviation (averaging 4.52% error from the GPS destination), confirming precise setpoint synchronization between the Pixhawk EKF2 estimator and the companion computer.

TABLE III: End-to-End Pipeline Latency Breakdown

Computational Pipeline Stage	Average Duration (ms)
USB Camera Frame Capture (GStreamer)	41.87
OpenCV Frame Conversion & Resize	22.15
YOLOv8n ONNX Tensor Inference	59.08
Non-Maximum Suppression (NMS)	15.42
Proportional Navigation Calculation	8.35
MAVLink Serialization & UART Transfer	41.17
Total End-to-End Latency	**188.04 ms**

G. Failsafe Safety and Persistent Scouting

The three-layer failsafe architecture effectively neutralized barometric altitude drift anomalies. During battery depletion tests (at 13-14% capacity), the drone successfully executed the Pre-RTL Safety Climb for 3.0 seconds, gaining 1.6m - 2.4m of elevation before initiating the horizontal RTL maneuver (Figure 8). The manual RC Override switch successfully terminated the autonomy script in all test cases.

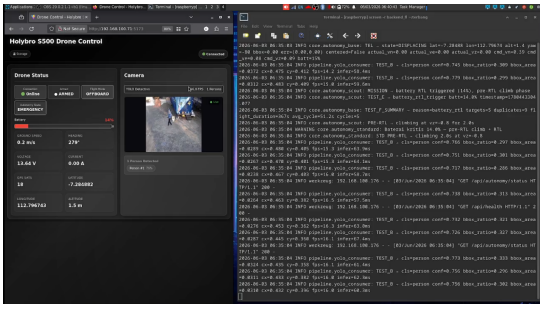


Fig. 8: Trigger activation of the RTL battery defense system.

The Persistent Scouting logic documented 11 distinct targets across three flights. Utilizing the Haversine formula with a 3-meter exclusion radius, the system successfully bypassed 19 duplicate detections without triggering RTL, proving the drone's capability to continue area observation efficiently. The average tracking cycle duration was 51-56 seconds per subject. During these intensive operations, the Raspberry Pi 5 maintained an average CPU utilization of 144% (multi-core) and RAM usage of 600 MB. Zero incidences of thermal throttling were observed, validating the headless OS and passive cooling design.

(a) Target A Documented

(b) Target B Documented

(c) Target C Documented

(d) Target D Documented

Fig. 9: Visual proof of the Persistent Scouting scenario, demonstrating the drone's capability to continuously navigate and document multiple distinct targets within a single flight session without returning to launch.

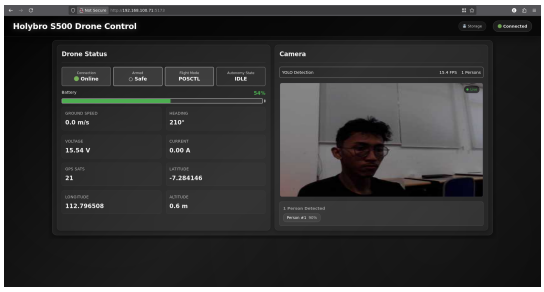


Fig. 10: Web interface of the real-time operator monitoring dashboard receiving 5 Hz WebSockets along with low latency UDP video captures.

VI. Discussion

While the developed system demonstrates robust autonomous SAR capabilities, several critical hardware and software trade-offs must be acknowledged:

- Model Resolution Trade-offs: Scaling down the YOLOv8n tensor to 320x320 pixels significantly improved inference latency, achieving 23 FPS on edge hardware. The reduction in matrix operations prevents memory bottlenecks on the Raspberry Pi 5. However, this resolution compromises long-range detection. The convolution filters lack sufficient pixel density to extract human contours beyond 15 meters, severely degrading tracking accuracy and forcing the drone to conduct scouting missions at closer, potentially riskier proximities to the terrain.
- Model Architecture Limitations: The baseline YOLOv8n bounding box detector was chosen over YOLOv8n-pose due to lower computational overhead and greater resistance to motion blur. While skeletal keypoint data could theoretically improve posture analysis for injured victims (e.g., detecting individuals lying down versus standing), the pose model proved too unstable for reliable dynamic targeting given the hardware constraints. The flickering of skeletal joints caused erratic proportional navigation feedback, rendering the flight path unstable.

- Camera Constraints: The reliance on a single fixed monocular RGB camera restricts operational capability. Without an active mechanical gimbal, pitch and roll corrections during flight momentarily disrupt the target alignment coordinate geometry. Every time the drone pitches forward to move, the camera tilts down, displacing the bounding box artificially. Furthermore, the standard optical system is incapable of operating in low-light or night-time conditions, which are often critical periods during SAR deployment when victims need immediate thermal discovery.
- Duplicate Exclusion Logistics: The GPS-based Haversine exclusion algorithm successfully prevents infinite loops on a single target by creating a 3-meter geospatial blackout radius. However, it lacks visual Person Re-Identification (Re-ID). Consequently, if multiple distinct victims are clustered within the same 3-meter radius—a common scenario in disaster evacuation points—the system may erroneously classify them as a single entity, ignoring the others. Future iterations must integrate algorithms like DeepSORT or OSNet to differentiate subjects based on visual clothing features rather than just coordinates.
- Deployment Communication: The system's video telemetry relies on a 4G/5G mobile router. In remote disaster zones with degraded or completely destroyed cellular infrastructure, the GCS video feed may suffer severe stuttering, packet loss, or complete failure. However, the true advantage of the edge computing architecture is that it guarantees the drone's autonomous search capability remains entirely unaffected by such signal loss. Even if the operator's screen goes black, the Raspberry Pi 5 will continue to execute the FSM, navigate the drone, and save victim documentation internally for post-flight retrieval.

VII. Conclusion

The integration of a Holybro S500 drone with a Pixhawk 6C and a Raspberry Pi 5 provides a highly effective prototype for autonomous Search and Rescue operations. Utilizing a headless OS, a 5A UBEC power supply, and an ONNX-quantized YOLOv8n model, the system executes real-time multi-threaded edge computing with an end-to-end latency below 200 ms. The Persistent Multi-Target Scouting state machine allows the drone to discover and document multiple victims continuously without repetitive RTL maneuvers. The Three-Layer Safety architecture, notably the Pre-RTL Safety Climb, effectively safeguards the vehicle against barometric drift. Future enhancements should incorporate infrared or thermal imaging payloads for night operations, along with visual re-identification models to improve target clustering accuracy.

References

[1] M. Lyu, Y. Zhao, C. Huang, and H. Huang, "Unmanned aerial vehicles for search and rescue: A survey," *Remote Sensing*, vol. 15, no. 13, p. 3266, 2023.

[2] Fact.MR, "Search and rescue drone market study," 2024.

[3] Badan Nasional Penanggulangan Bencana, *Data Bencana Indonesia 2024*. Jakarta, Indonesia: Pusat Data, Informasi, dan Komunikasi Kebencanaan BNPB, 2025.

[4] B. M. Sopha, A. M. S. Asih, and J. I. Agriawan, "Adopters and non-adopters of drones in humanitarian operations: An empirical evidence from a developing country," *Progress in Disaster Science*, vol. 21, p. 100314, 2024.

[5] Z. A. Elashaal, Y. H. Lamin, M. K. Elfandi, and A. A. Eluheshi, "Autonomous search and rescue drone," *Libyan Journal of Informatics*, vol. 1, no. 1, pp. 1–8, 2024.

[6] K. Jayalath and S. R. Munasinghe, "Drone-based autonomous human identification for search and rescue missions in real-time," in *Proc. 10th Int. Conf. Information and Automation for Sustainability (ICIAfS)*, Moratuwa, Sri Lanka, 2021.

[7] V. Varatharasan, A. S. S. Rao, E. Toutounji, J.-H. Hong, and H.-S. Shin, "Target detection, tracking and avoidance system for low-cost UAVs using AI-based approaches," in *Proc. Int. Workshop RED-UAS*, Cranfield, UK, 2019, pp. 142–149.

[8] R. Austin, *Unmanned Aircraft Systems: UAV Design, Development and Deployment*. John Wiley & Sons, 2021.

[9] Holybro Robotics, *S500 Quadcopter Frame Technical Manual*. Holybro, 2023.

[10] Raspberry Pi Foundation, *Raspberry Pi 5 Technical Overview*. Raspberry Pi Ltd., 2024.

[11] H. Zhou, J. Wang, Z. Liu, and Y. Li, "SOD-YOLOv8: Enhancing YOLOv8 for small object detection," *Sensors*, vol. 24, no. 19, p. 6209, 2024.

[12] MAVLink Development Team, *Micro Air Vehicle Link Protocol Specification*. MAVLink.io, 2024.

[13] MAVSDK Project, "MAVSDK — A MAVLink SDK for drones, cameras and ground systems," https://mavsdk.mavlink.io/, 2025.

[14] A. Kumar, S. Chaudhary, S. Sangal, and R. Dhama, "Review on computer vision using OpenCV," *Int. J. Computer Applications Technology and Research*, vol. 3, no. 12, pp. 756–761, 2024.

[15] D. McNulty, "A review of Li-ion batteries for autonomous mobile robots," *Sensors and Actuators A: Physical*, vol. 339, p. 113567, 2022.

[16] W. Shi, J. Cao, Q. Zhang, Y. Li, and L. Xu, "Edge computing: Vision and challenges," *IEEE Internet of Things Journal*, vol. 3, no. 5, pp. 637–646, 2016.